AWS Account Security Runbook by simeon siaka

Version 1.0 — Built during AWS Cloud Accelerator, Week 2.

Section 1:

AWS new account setup checklist with progress tracking

Root account security

- [ ] Enable MFA on the root account critical

  Use a hardware key or authenticator app. Never use root for day-to-day tasks.

- [ ] Delete root access keys critical

  Root access keys should never exist. Remove them immediately if present.

- [ ] Set a strong root account password

  Use a unique, randomly generated password stored in a password manager.

- [ ] Set alternate contacts (billing, operations, security)

  Ensures the right people receive AWS notifications — not just the root email.

IAM & access management

- [ ] Create an IAM admin user (or use IAM Identity Center) IAM

  Do all work as this user, not root. Prefer IAM Identity Center for multi-account setups.

- [ ] Enable MFA for all IAM users critical

  Enforce via IAM policy — deny all actions unless MFA is active.

- [ ] Set an IAM password policy IAM

  Minimum length 14+, require complexity, prevent reuse of last 24 passwords.

- [ ] Apply least-privilege permissions from day one IAM

  Never attach AdministratorAccess to service roles. Use IAM Access Analyzer to verify.

Billing & cost management

- [ ] Enable billing alerts and set a budget free

  Create a zero-spend or monthly budget in AWS Budgets. Alert at 80% and 100% of threshold.

- [ ] Enable Cost Explorer free

  Activate it early so historical data starts accumulating for trend analysis.

- [ ] Grant IAM users access to billing console

  By default, only root sees billing. Enable IAM access to billing in account settings.

Logging & monitoring

- [ ] Enable AWS CloudTrail in all regions critical

  Create a multi-region trail. Store logs in an S3 bucket with object lock enabled.

- [ ] Enable AWS Config

  Records resource configuration history. Essential for compliance and change auditing.

- [ ] Enable Amazon GuardDuty critical

  Intelligent threat detection. 30-day free trial. Enable in every region you use.

- [ ] Enable AWS Security Hub

  Aggregates findings from GuardDuty, Inspector, and Config into one security dashboard.

Networking

- [ ] Delete or lock down the default VPC

  The default VPC has permissive settings. Remove it or restrict its security groups immediately.

- [ ] Enable VPC Flow Logs for your primary VPC

  Captures accepted and rejected traffic. Store in S3 or CloudWatch Logs.

Region & service hygiene

- [ ] Opt out of unused regions using SCP or IAM deny

  Reduces attack surface. Prevents accidental resource creation in distant regions.

- [ ] Enable S3 Block Public Access at the account level critical

  Prevents any S3 bucket from accidentally being made public — account-wide safeguard.

- [ ] Tag resources with a consistent tagging strategy

  Set up Tag Policies via AWS Organizations. Suggested: Owner, Environment, Project, CostCenter.

Section 2:

Eight IAM best practices for AWS accounts

IAM best practices — 8 rules

01

Never use the root account for day-to-day work

Root has unrestricted access to everything and cannot be restricted by SCPs or IAM policies. Reserve it solely for tasks that explicitly require it: closing the account, changing the support plan, or restoring IAM access.

critical

Do

Create a dedicated IAM admin user or use IAM Identity Center for all daily operations.

Don't

Log in as root to deploy resources, manage S3, or run CLI commands.

02

Grant least-privilege permissions — nothing more

Start with zero permissions and add only what is needed. Avoid *:* wildcards. Use IAM Access Analyzer to identify overly broad policies and validate them before attaching.

Critical design principle

Do

Write scoped policies: specific actions, specific resources, specific conditions.

Don't

Attach AdministratorAccess to service roles or Lambda functions.

03

Enforce MFA for all human users

Add an IAM policy that denies all actions unless aws:MultiFactorAuthPresent is true. This catches any session where MFA was skipped, regardless of how the user authenticated.

Critical enforce via policy

Do

Use a hardware security key (YubiKey) or TOTP app. Apply the MFA-deny policy account-wide.

Don't

Rely on users to voluntarily enable MFA — enforce it programmatically.

04

Use IAM roles instead of long-lived access keys

EC2 instances, Lambda functions, ECS tasks, and CI/CD pipelines should assume roles — not use static access keys stored in environment variables or config files. Roles issue temporary credentials that auto-rotate.

Critical design principle

Do

Assign instance profiles to EC2, execution roles to Lambda, task roles to ECS.

Don't

Hardcode AWS_ACCESS_KEY_ID in code, .env files, or Docker images.

05

Rotate and audit access keys regularly

If long-lived keys must exist (e.g. third-party integrations), rotate them every 90 days. Use the IAM credential report to find keys older than 90 days, keys that have never been used, and users with multiple active keys.

Audit enforce via policy

Do

Run aws iam generate-credential-report monthly. Deactivate unused keys.

Don't

Leave inactive access keys active — they are a silent attack vector.

06

Use groups and roles to assign permissions — never attach policies to individual users

Attaching policies directly to users creates a management nightmare at scale and makes auditing nearly impossible. Use IAM groups for human users and roles for everything else.

design principle

Do

Create groups like Developers, ReadOnly, DataScientists. Assign policies to the group.

Don't

Attach AmazonS3FullAccess or any managed policy directly to a user.

07

Set permission boundaries on delegated roles

When developers or automation can create IAM roles, attach a permission boundary that caps the maximum permissions any role they create can have. This prevents privilege escalation — a developer cannot create a role that exceeds their own permissions.

enforce via policy design principle

Do

Define a boundary policy and require it on all programmatically created roles via SCP.

Don't

Allow developers unrestricted iam:CreateRole or iam:AttachRolePolicy.

08

Audit IAM continuously with Access Analyzer and CloudTrail

IAM Access Analyzer flags resources shared with external principals. CloudTrail logs every iam: API call. Set CloudWatch alarms for high-risk actions like CreateUser, AttachUserPolicy, and DeleteTrail.

audit

Do

Enable Access Analyzer per region. Review its findings weekly. Alert on root login.

Don't

Treat IAM as a set-and-forget config — permissions drift over time.

Section 3:

Step-by-step AWS credential compromise response playbook with progress tracking

Stop. Do not delete the keys yet — disabling first preserves evidence. Work fast but in order.

12 steps complete

Phase 1 — Immediate containment (do this in minutes)

01

- [ ] Disable the compromised credentials immediately first

  For IAM access keys: set status to Inactive — do not delete yet. For an IAM user: add an explicit Deny-all policy inline. This cuts access without destroying evidence in CloudTrail.

  aws iam update-access-key --access-key-id AKIA... --status Inactive

02

- [ ] Invalidate all active sessions for the principal

  Attach the AWS-managed AWSRevokeOlderSessions policy to the user, or update the role's trust policy with a date condition. This terminates any STS tokens already in flight.

  aws iam attach-user-policy --user-name ... --policy-arn arn:aws:iam::aws:policy/AWSRevokeOlderSessions

03

- [ ] If root credentials are involved — take emergency action critical

  Log in as root immediately and delete the compromised root access keys. Enable root MFA if not already active. Change the root password. Contact AWS Support — they have escalation paths for root account compromise.

Phase 2 — Assess the blast radius (within the first hour)

04

- [ ] Query CloudTrail for all actions taken with the credential investigate

  Search for API calls starting from the estimated time of compromise. Look at every service — not just IAM. Pay special attention to: CreateUser, AttachUserPolicy, RunInstances, GetSecretValue, ListBuckets.

  aws cloudtrail lookup-events --lookup-attributes AttributeKey=Username,AttributeValue=<user>

05

- [ ] Check for new or modified IAM entities investigate

  Attackers almost always create a backdoor user or role immediately after gaining access. List all IAM users, roles, and access keys created after the compromise window. Cross-reference with CloudTrail.

aws iam generate-credential-report && aws iam get-credential-report

06

- [ ] Audit all regions for unexpected resources

  Cryptomining and data exfiltration infrastructure is often spun up in regions you never use. Check EC2 instances, Lambda functions, S3 buckets, and RDS instances in every region — not just your primary one.

  aws ec2 describe-instances --region <each-region> --query 'Reservations[*].Instances[*].[InstanceId,State.Name,LaunchTime]'

07

- [ ] Check Secrets Manager and Parameter Store for access investigate

  If the credential had Secrets Manager or SSM access, assume all secrets it could read are now compromised. List recent GetSecretValue events in CloudTrail to identify exactly what was read.

Phase 3 — Eradicate and recover

08

- [ ] Delete all attacker-created resources and backdoor accounts eradicate

  Remove any IAM users, roles, access keys, or policies created during the compromise window. Terminate unknown EC2 instances. Delete unknown S3 buckets — check their contents first for exfiltrated data.

09

- [ ] Rotate all potentially exposed secrets and credentials

  Rotate every secret the compromised principal could have accessed: database passwords, API keys, Secrets Manager values, Parameter Store SecureStrings, and any other credentials stored in the environment. Assume they are all known to the attacker.

10

- [ ] Delete the original compromised key — then issue a replacement recover

  Now that the investigation is complete, delete the disabled key. Issue a new key or role with tighter permissions than the original. Update any services, CI/CD systems, or scripts that used the old credential.

  aws iam delete-access-key --access-key-id AKIA…

Phase 4 — Notify and prevent recurrence

11

- [ ] Notify affected stakeholders and assess regulatory obligations notify

  If PII, financial data, or health data was in S3 or RDS and may have been accessed, you may have a legal notification obligation (GDPR 72hr, HIPAA, PCI-DSS). Engage legal counsel. File an AWS abuse report if attacker infrastructure is still visible.

- [ ] Conduct a post-incident review and close the exposure vector prevent

  Answer: how did this key get exposed? (leaked in git, hardcoded in a Docker image, overly broad CI permissions?) Fix the root cause. Run IAM Access Analyzer. Enable GuardDuty if it wasn't already on. Consider AWS Config rules to alert on long-lived key creation.

Section 4:

AWS monthly security review checklist across six domains

28 complete

IAM & access 0 / 6

- [ ] Run IAM credential report critical

  Flag keys unused for 90+ days, users with no MFA, and multiple active keys per user.

- [ ] Review IAM Access Analyzer findings

  Resolve any resources shared with external AWS accounts or the public.

- [ ] Deactivate or delete stale access keys

  Rotate any key older than 90 days that is still active and in use.

- [ ] Audit users who left the team

  Disable or delete IAM users for people who no longer need access.

- [ ] Review service role permissions for drift

  Check Lambda, EC2, and ECS roles for permissions added outside of IaC.

- [ ] Verify root account has no active access keys

  Should be zero. If any exist, delete them immediately.

Logging & monitoring 0 / 5

- [ ] Confirm CloudTrail is active in all regions critical

  Check that the multi-region trail is enabled and writing to S3 with no delivery errors.

- [ ] Review GuardDuty findings

  Triage all High and Medium severity findings. Archive resolved ones — never ignore them.

- [ ] Review Security Hub findings

  Check the AWS Foundational Security Best Practices standard score. Target 90%+.

- [ ] Verify CloudWatch alarms are firing correctly

  Test alarms for root login, failed console logins, and MFA disable events.

- [ ] Check CloudTrail log S3 bucket integrity

  Verify log file validation is enabled and no log files have been deleted or modified.

Networking & exposure 0 / 5

- [ ] Audit security groups for 0.0.0.0/0 ingress critical

  Any SG allowing unrestricted inbound on SSH (22), RDP (3389), or databases (3306, 5432) is a critical finding.

- [ ] Check S3 bucket public access settings

  Verify the account-level Block Public Access is still on. Review any intentional exceptions.

- [ ] Review RDS and ElastiCache public accessibility

  No database instance should have PubliclyAccessible set to true unless explicitly required.

- [ ] Scan for exposed EC2 instances (no SSM agent)

  Instances reachable only via key pair SSH are harder to audit. Migrate to SSM Session Manager.

- [ ] Review VPC peering and PrivateLink connections

  Confirm all active peering connections and endpoints are still required and correctly scoped.

Data protection 0 / 4

- [ ] Verify S3 encryption at rest on sensitive buckets review

  Confirm SSE-KMS is applied to buckets holding PII, financial, or secrets data.

- [ ] Check RDS automated backup retention and encryption

  Backups should be encrypted and retained for at least 7 days. Verify restore was tested this quarter.

- [ ] Audit KMS key policies and grants

  Review who can use and administer each CMK. Remove stale grants from deleted roles or users.

- [ ] Review Secrets Manager secret rotation status

  Check that auto-rotation is enabled on all database credentials and API keys stored in Secrets Manager.

Cost & resource hygiene 0 / 4

- [ ] Review Cost Explorer for anomalies review

  Unexpected spend spikes — especially in unfamiliar regions — are a common indicator of compromise.

- [ ] Audit running resources in all regions

  Check for EC2, RDS, NAT Gateways, and Elastic IPs in regions you don't actively use.

- [ ] Check AWS Budgets thresholds are still accurate

Adjust budget limits if spend patterns have grown. Stale budgets stop being useful alerts.

- [ ] Remove unattached EBS volumes and unused snapshots

  These accumulate quietly and carry both cost and data residency risk if left indefinitely.

Compliance & config 0 / 4

- [ ] Review AWS Config non-compliant rules compliance

  Prioritise rules tagged as security-related. Each non-compliant rule represents a config drift from baseline.

- [ ] Check for unpatched EC2 instances via Inspector

  Review Amazon Inspector findings for critical and high CVEs on running instances and container images.

- [ ] Verify SCPs are in effect on all OUs compliance

  Confirm that deny-region, deny-root-actions, and require-encryption SCPs are still attached correctly.

- [ ] Review Trusted Advisor security recommendations

  Even on free tier, Trusted Advisor flags MFA on root, security group exposure, and S3 bucket permissions.